

Structures de données authentifiées : arbres de Merkle et preuves d'inclusion

Il est rappelé que le jury n'exige pas une compréhension exhaustive du texte. Vous êtes laissé(e) libre d'organiser votre présentation comme vous l'entendez. Il vous est conseillé de mettre en valeur vos connaissances et votre culture scientifique à partir du fil conducteur constitué par le texte. Les *suggestions de développement*, en fin de texte, ne sont là que pour vous aider ; vous ne devez pas vous sentir tenu(e) de les suivre, ni de les traiter toutes. Le jury appréciera que votre exposé soit accompagné d'exemples et d'expériences traités sur ordinateur.

1 Un problème de vérification à faible coût

Les registres distribués (*blockchains*) comme Bitcoin ou Ethereum maintiennent, sur des milliers de machines indépendantes, un état commun de très grande taille. Une opération y revient sans cesse : *convaincre* un participant qu'une donnée précise (une transaction, un solde, une adresse figurant sur une liste d'autorisation) appartient bien à un ensemble donné, *sans* lui transmettre l'ensemble entier ni exiger qu'il le recalcule. Le coût de stockage et de communication « sur la chaîne » étant élevé, on cherche des *certificats* de taille aussi petite que possible.

Formulons le problème. On dispose d'une suite finie de données $d_0, d_1, \dots, d_{n-1}$ (les *feuilles*). Un acteur, le *prouveur*, publie un unique condensé R résumant toute la suite. Plus tard, il veut prouver à un *vérifieur* que « d_i figure à la position i dans la suite ayant produit R », en fournissant un certificat de taille minimale. Le vérifieur ne connaît que R et la donnée d_i présentée ; il doit pouvoir *rejeter* toute donnée qui ne figurait pas réellement dans la suite.

L'objet de ce texte est la solution devenue standard, l'*arbre de Merkle* (R. Merkle, 1979), qui atteint des certificats de taille $O(\log n)$. Elle repose entièrement sur une brique cryptographique, la *fonction de hachage résistante aux collisions*, que nous étudions d'abord. Le fil conducteur mêle arithmétique modulaire, dénombrement, un argument probabiliste (le *paradoxe des anniversaires*) et une preuve de correction par récurrence.

2 Fonctions de hachage et résistance aux collisions

Définition 1. Une *fonction de hachage* est une application $H : \mathcal{M} \rightarrow \{0, 1, \dots, M-1\}$ d'un ensemble de messages $\mathcal{M}$ (potentiellement infini) vers un ensemble fini de M valeurs. Une *collision* est un couple de messages distincts $m \neq m'$ tels que $H(m) = H(m')$.

Dans les implémentations réelles on prend $M = 2^b$ (par exemple $b = 256$ pour Keccak-256, utilisée par Ethereum). Pour les besoins des *expériences*, on pourra travailler avec la fonction de combinaison arithmétique jouet

$$h(a, b) = (31a + b) \bmod p, \quad p = 1000003 \text{ premier}, \quad (1)$$

qui joue le rôle d'un « hachage de deux valeurs en une ».

L'observation fondamentale est que les collisions *existent* nécessairement.

Proposition 1 (Principe des tiroirs). *Si $\mathcal{M}$ est de cardinal strictement supérieur à M (en particulier s'il est infini), alors H n'est pas injective : elle admet au moins une collision.*

Proof. Si H était injective, elle réaliserait une bijection de $\mathcal{M}$ sur une partie de l'ensemble d'arrivée, de cardinal au plus M ; on aurait $|\mathcal{M}| \leq M$, ce qui contredit l'hypothèse. Il existe donc $m \neq m'$ avec $H(m) = H(m')$. $\square$

La sécurité ne peut donc pas reposer sur l'absence de collisions. On demande seulement qu'elles soient *calculatoirement introuvables*.

Définition 2. H est dite *résistante aux collisions* s'il n'existe pas d'algorithme efficace (en temps polynomial) produisant, avec probabilité non négligeable, une collision de H .

C'est une hypothèse *cryptographique*, non un théorème : elle porte sur la difficulté algorithmique, pas sur l'inexistence. Quantifions la difficulté de trouver une collision par *recherche aveugle*, en modélisant H comme un tirage uniforme dans $\{0, \dots, M-1\}$ (modèle de l'*oracle aléatoire*).

Proposition 2 (Borne des anniversaires). *On tire k valeurs indépendantes et uniformes dans un ensemble à M éléments. La probabilité q_k qu'elles soient deux à deux distinctes vaut*

$$q_k = \prod_{i=0}^{k-1} \left(1 - \frac{i}{M}\right), \quad \text{et} \quad q_k \leq \exp\left(-\frac{k(k-1)}{2M}\right).$$

En particulier, la probabilité d'au moins une collision dépasse $1/2$ dès que $k \gtrsim 1,177 \sqrt{M}$.

Proof. Le i -ième tirage évite les i valeurs déjà obtenues avec probabilité $1 - i/M$; par indépendance, q_k est le produit annoncé. En utilisant $1 - x \leq e^{-x}$ pour tout x , on majore $q_k \leq \prod_{i=0}^{k-1} e^{-i/M} = \exp\left(-\frac{1}{M} \sum_{i=0}^{k-1} i\right) = \exp\left(-\frac{k(k-1)}{2M}\right)$. La probabilité de collision $1 - q_k$ dépasse $1/2$ lorsque $q_k \leq 1/2$, ce qui est assuré dès que $\frac{k(k-1)}{2M} \geq \ln 2$, soit $k \gtrsim \sqrt{2 \ln 2} \sqrt{M} \approx 1,177 \sqrt{M}$. $\square$

Ainsi une fonction sur b bits n'offre qu'une sécurité de l'ordre de $2^{b/2}$ opérations contre les collisions : c'est pourquoi on choisit $b = 256$ (soit $\approx 2^{128}$, hors de portée). Le cas familier $M = 365$ redonne le *paradoxe des anniversaires* : 23 personnes suffisent à rendre une coïncidence plus probable qu'improbable. On pourra vérifier ces seuils par une **simulation de Monte-Carlo** :

```

1 import random
2 def collision_probable(M, k, essais=20000):
3     succes = 0
4     for _ in range(essais):
5         vus = set(); coll = False
6         for _ in range(k):
7             v = random.randrange(M)
8             if v in vus: coll = True; break
9             vus.add(v)
10        succes += coll
11    return succes / essais
12 # collision_probable(365, 23) ~ 0.50
13 # collision_probable(10**6, 1178) ~ 0.50

```

3 Arbres de Merkle

L'idée est de *hacher par niveaux*. Les feuilles $d_0, \dots, d_{n-1}$ (déjà hachées, vues comme des entiers) forment le niveau 0 ; on remplace chaque paire consécutive par son haché combiné $h(\cdot, \cdot)$, obtenant le niveau 1, deux fois plus court ; on itère jusqu'à ne conserver qu'une valeur, la *racine* R . Lorsqu'un niveau a une longueur impaire, on *duplique* sa dernière valeur avant l'appariement.

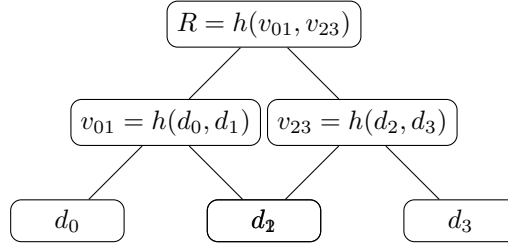


Figure 1 — Arbre de Merkle sur quatre feuilles.

Avec la fonction jouet (1) et les feuilles $[1, 2, 3, 4]$ on obtient $v_{01} = h(1, 2) = 33$, $v_{23} = h(3, 4) = 97$, puis $R = h(33, 97) = 1120$. Voici la construction :

```

1 def h(a, b):
2     return (31 * a + b) % 1000003
3
4 def racine(feuilles):
5     niveau = list(feuilles)
6     while len(niveau) > 1:
7         if len(niveau) % 2 == 1:
8             niveau.append(niveau[-1]) # duplication si impair
9             niveau = [h(niveau[i], niveau[i+1])
10                      for i in range(0, len(niveau), 2)]
11     return niveau[0]
12 # racine([1, 2, 3, 4]) == 1120

```

Proposition 3 (Complexité). *Le calcul de la racine de n feuilles effectue $O(n)$ évaluations de h , en temps $O(n)$ et espace $O(n)$. L'arbre a une hauteur $H(n) = \lceil \log_2 n \rceil$.*

Proof. Notons $t_0 = n$ et $t_{k+1} = \lceil t_k/2 \rceil$ la taille du niveau $k+1$; ce niveau demande $\lceil t_k/2 \rceil$ évaluations de h . Comme $t_{k+1} \leq t_k/2 + 1$, une récurrence donne $t_k \leq n 2^{-k} + 2$, donc $t_k \leq 2$ dès que $k \geq \log_2 n$: la hauteur est $\lceil \log_2 n \rceil$ et il y a $O(\log n)$ niveaux. Le nombre total d'évaluations vaut $\sum_{k \geq 1} t_k \leq \sum_{k \geq 1} (n 2^{-k} + 2) \leq n + O(\log n) = O(n)$. $\square$

4 Preuves d'inclusion et sécurité

Définition 3. Une *preuve d'inclusion* de la feuille d'indice i est la suite des couples (frère, côté) rencontrés en remontant de cette feuille jusqu'à la racine, où le côté indique si le frère se situe à gauche (L) ou à droite (R). Pour la vérifier, on repart de la feuille présentée et l'on recombine successivement par h avec chaque frère selon son côté ; la feuille est *acceptée* si l'on retrouve la racine R .

Sur la Figure 1, la preuve de d_0 est $[(d_1, R), (v_{23}, R)]$: le vérifieur calcule $h(d_0, d_1) = v_{01}$ puis $h(v_{01}, v_{23}) = R$. La longueur d'une preuve est le nombre de niveaux, soit $H(n) = \lceil \log_2 n \rceil = O(\log n)$: pour un milliard de feuilles, une trentaine de valeurs suffisent, à comparer à la transmission des n feuilles. C'est tout l'intérêt de la construction.

Reste la propriété de sécurité : un prouveur malhonnête ne doit pas pouvoir faire accepter une feuille absente. C'est ici que sert l'hypothèse de résistance aux collisions.

Théorème 1 (Correction, ou *soundness*). *Supposons h résistante aux collisions. Si la vérification d'une preuve d'inclusion pour une valeur x aboutit à la racine authentique R , alors, sauf à exhiber une collision de h , la valeur x figure effectivement parmi les feuilles ayant produit R .*

Proof. Récurrence sur la hauteur H de l'arbre. Si $H = 0$, l'arbre est réduit à sa racine, qui est la feuille ; la preuve est vide et n'est acceptée que si $x = R$, donc x est bien la feuille. Supposons le résultat acquis à la hauteur H et considérons un arbre de hauteur $H + 1$, de racine $R = h(g, d)$ où g, d sont les deux valeurs de niveau 1 réellement présentes. La vérification calcule d'abord $r' = h(x, f)$ (ou $h(f, x)$) avec le premier frère f , puis poursuit avec le reste de la preuve : c'est une vérification d'inclusion de r' dans l'arbre des nœuds de niveau 1, de hauteur H . Par hypothèse de récurrence, ou bien r' est l'un des nœuds authentiques de ce niveau, ou bien on a exhibé une collision. Dans le premier cas, $r' \in \{g, d\}$ et $r' = h(x, f)$; comme r' possède déjà l'antécédent authentique issu de la construction, l'égalité $h(x, f) = r'$ fournit soit *le même* couple (et alors x est la feuille voulue), soit un couple *différent* de même image, c'est-à-dire une collision de h . En l'absence de collision calculable, x figure donc bien parmi les feuilles. $\square$

Remarque 1. La sécurité de tout l'édifice se réduit ainsi à celle de h : forger une fausse preuve d'inclusion, c'est calculer une collision. Cette *réduction* est l'exacte raison pour laquelle on exige $b = 256$ bits (Proposition 2).

5 Aspects dynamiques et prolongements

Mises à jour. Lorsqu'une seule feuille change, seuls les nœuds situés sur le chemin de cette feuille à la racine sont affectés : il y en a $H(n) = O(\log n)$. En conservant les nœuds inchangés, une mise à jour de la racine ne coûte donc que $O(\log n)$ évaluations de h , et non $O(n)$. Cette propriété est essentielle pour les « arbres de Merkle persistants » qui indexent l'état d'Ethereum.

Au-delà. Les arbres de Merkle sont un cas particulier d'*accumulateur cryptographique*. Ethereum emploie une variante, l'arbre de *Patricia-Merkle*, mêlant arbre préfixe et hachage ; le projet *Verkle* remplace le hachage binaire par des *engagements polynomiaux* (schémas de type KZG) afin de réduire encore la taille des preuves, de $O(\log n)$ à $O(1)$. Enfin, la vérification « à faible coût » illustrée ici est l'idée directrice des *preuves à divulgation nulle de connaissance* et des *rollups* : faire vérifier un calcul massif à moindre frais.

Suggestions pour le développement

Les pistes suivantes sont largement indépendantes ; on n'en traitera qu'une partie.

- Démontrer soigneusement la Proposition 2 et discuter l'équivalent asymptotique du seuil $k^*(M)$; comparer aux mesures de la simulation de Monte-Carlo (cas $M = 365$, $M = 10^6$, $M = 2^{32}$).
- Implémenter la construction de l'arbre, la génération et la vérification des preuves d'inclusion ; mesurer expérimentalement la taille des preuves en fonction de n et confronter à $\lceil \log_2 n \rceil$.
- Détailler la preuve du Théorème 1 et expliciter la *réduction* : comment une fausse preuve fournit-elle algorithmiquement une collision ?
- Étudier le coût amorti des mises à jour dynamiques ; programmer une version en $O(\log n)$ conservant les nœuds inchangés et la comparer au recalcul complet.
- Analyser l'arithmétique de la fonction jouet (1) : pour quels choix de multiplicateur et de module p obtient-on une bonne dispersion ? Relier au hachage polynomial et aux corps $\mathbb{Z}/p\mathbb{Z}$.

- Comparer, en taille de preuve et en coût de vérification, l'arbre de Merkle binaire, un arbre k -aire, et un engagement vectoriel de type Verkle.
- Modéliser une *liste d'autorisation* (*whitelist*) d'adresses par sa racine de Merkle ; estimer l'économie de stockage « sur la chaîne ».
- Situer le problème de sécurité dans le cadre des *fonctions à sens unique* et des hypothèses de la cryptographie moderne.

Références

1. R. C. Merkle, *A digital signature based on a conventional encryption function*, CRYPTO '87.
2. A. M. Antonopoulos, G. Wood, *Mastering Ethereum*, O'Reilly, 2^e éd. (chapitres « Cryptography », « The Ethereum Virtual Machine », « Decentralized Applications »).
3. J. Katz, Y. Lindell, *Introduction to Modern Cryptography*, CRC Press.
4. D. Boneh, V. Shoup, *A Graduate Course in Applied Cryptography* (chapitre sur les fonctions de hachage et la borne des anniversaires).